Vibe Coding Is Dead: A New York Engineer's Field Notes

The rain had just stopped over DUMBO when my phone buzzed with yet another demo video. Cobblestones glistened under the Manhattan Bridge, tourists lined up for the classic skyline photo, and there I was, thumb hovering over a LinkedIn clip of a fully playable racing game that someone had supposedly built over a weekend by chatting with an AI.

Comments piled up underneath within minutes. "The future of software engineering is here." "Junior developers are finished." "Why hire developers at all anymore?" I put the phone away and watched a street magician set up a folding table near the waterfront instead.

The magician pulled a silk scarf out of thin air, and a circle of kids gasped. Parents smiled the way people smile when they know they are being fooled and enjoy it anyway. Nobody in that crowd believed the scarf had materialized from nothing, and nobody needed to believe it. The trick was the product, and the wonder was the point. A good crowd, an honest deception, everybody goes home happy.

That is the entire business model of vibe coding, and I say that as someone who has shipped production software for more than a decade. A magic shop sells you the trick: the scarf, the collapsing wand, the deck that always finds your card. What it never sells you is the 10,000 hours of practice that make a magician. You walk out feeling like a magician for exactly 1 evening, performing for friends who politely applaud, and then the trick goes in a drawer with the other toys.

Walk the side streets off Times Square and you will still find those shops, running the same business model for nearly 100 years. Sell the dream of being a magician. Keep the reality of earning the title. The AI coding industry reinvented that shop with better

lighting and a subscription plan, and the scarf still comes out of thin air. Applause still follows. Nobody walks out a magician.

Now ask the question nobody in the demo videos asks. If vibe coding is dead or alive, who profits from the answer? The companies selling AI coding tools, obviously. They sell the dream that software engineering is now a conversation, that anyone can skip the hard parts, that teams can hire developers less or never hire developers at all because the machine does the work. The pitch is seductive precisely because it flatters the buyer. You are not cutting corners, the story goes. Instead you are riding the future.

And yet those same companies employ some of the largest engineering organizations on the planet, paying top-of-market salaries to thousands of software engineers who show up every day to write code by hand, review each other's work, argue about architecture, and carry pagers. Funny, that. The sellers of automated software engineering run their own engineering the old-fashioned way, and they show no sign of stopping.

Here is the question I keep asking, and I have never received a straight answer. If these AI coding tools could genuinely do software engineering and cut costs, why don't the companies selling them cut their own payroll before anyone else's? Why does every major AI lab still run enormous teams of human engineers building the very products that supposedly replace engineers? The silence that follows that question tells you everything the marketing never will.

Let me be direct about what I believe, because this is an opinion piece and I will own it as opinion. In my opinion, the current generation of AI coding products does not do what the marketing claims. It generates impressive demos. Engineering software is a different matter entirely. There is a canyon between those 2 things, and the entire vibe coding economy lives inside that canyon, selling maps of a bridge that was never built.

The phrase itself is barely 1 year old. Computer scientist Andrej Karpathy coined "vibe coding" in February 2025 to describe surrendering to the vibes and letting a large language model write the code while you supervise loosely. The term spread so fast

that Collins Dictionary named it the Word of the Year for 2025. You can read the full documented history, with sources, on Wikipedia.

What followed was a flood of demos. My feeds filled with AI-generated games, dashboards, and clones of famous apps, each presented as proof that software engineering had been automated overnight. But notice what nobody does with these demos: nobody sells them. If a weekend-built racing game were a real product, someone would put a price tag on it, support it, patch it, and answer customer emails at 3 AM. Instead the games stay in the feed, collecting likes, because a demo is a magic trick and a product is a profession. Characterize the flood however you like, but do not mistake it for a market.

The data backs up the skepticism, and it comes from sources the AI industry itself respects. In July 2025, the research group METR ran a randomized controlled trial with professional developers across 246 coding tasks. Developers using AI tools finished 19% slower than those working without them, even though the developers had predicted beforehand that the tools would make them 24% faster, and still believed afterward that the tools had made them 20% faster. Let that sink in. The tools felt fast and measured slow. Perception and stopwatch disagreed, and the stopwatch does not care about vibes.

Stack Overflow's 2024 developer survey found that 45% of professional developers believe AI tools are bad or very bad at handling complex tasks. The same survey found that 70% of professional developers do not perceive AI as a threat to their jobs. Developers are not afraid of AI coding tools. They are disappointed by them, which is considerably worse for the sales pitch. You can read the survey results directly on Stack Overflow.

The scale of the experiment running on other people's money is worth stating plainly. In March 2025, Y Combinator reported that 25% of startups in its Winter 2025 batch had codebases that were 95% AI-generated. Read that again slowly. A quarter of a prestige accelerator batch shipped products where nearly every line was written by a model, reviewed lightly if at all, and then handed to customers. Some of those startups

will succeed, and their success will be cited as proof that vibe coding works. Most will fail quietly, and their failures will be filed under "market conditions" rather than engineering reality. Survivorship bias is doing a lot of heavy lifting in this debate.

By mid-2025, working engineers had a name for the aftermath: vibe slop. Engineers behind professional coding tools warned publicly of a looming crisis, arguing that companies were trading near-term productivity for longer-term problems, including buggy software, service outages, security vulnerabilities, and increased technical debt. 1 engineer's summary stuck with me: infrastructure falling apart, software buggier than before, a game the industry can play for a few more months or years before it catches up. That is not a critic speaking; it is a practitioner describing the bill coming due.

Then there is the code quality problem, which is the problem nobody wants to discuss on stage. A December 2025 analysis of 470 open-source pull requests found that AI co-authored code contained roughly 1.7 times more major issues than human-written code, with security vulnerabilities running 2.74 times higher. AI-generated code looks confident and reads smoothly, which makes the bugs harder to spot, not easier. Confidence without comprehension is the most expensive kind of wrong, because it bills by the hour while it misleads.

None of this surprises anyone who has done real software engineering. Writing code was never the hard part of the job. The hard part is knowing which code to write, which to delete, and which will still make sense to a tired engineer at 2 AM during an outage 3 years from now. AI coding tools are excellent at producing text that resembles code. Software engineering is the discipline of producing systems that survive contact with reality. Those are different skills, and only 1 of them can be faked by a demo.

Consider where the money actually sits. Down in the Financial District, banks move trillions through systems where 1 misplaced decimal is a career-ending event. Ask yourself whether any of those institutions run AI-generated code anywhere near the ledger. They do not, and they will tell you why over drinks: the demo is not the product, and money does not run on vibes. When the stakes are real, everyone suddenly rediscovers the value of engineers who understand the system.

I keep coming back to Penn and Teller. Their long-running stage show includes a segment where amateur magicians try to fool them, and a magician earns real respect in that community only by fooling 2 professionals who know every trick in the book. That is the bar that matters. A trick that fools your friends is a toy. Fooling Penn and Teller, 2 professionals who know every trick in the book, is craft. You can see how seriously they take the difference on their official site.

Vibe coding has not fooled Penn and Teller. It has fooled the audience, which was always the easier trick. Every demo I have watched follows the same script: a charismatic presenter, a greenfield project with zero legacy constraints, a happy path with no edge cases, and a big reveal met with applause. Take that same workflow into a 15-year-old codebase with 4,000,000 lines, regulatory requirements, and 200 engineers who all have opinions, and the vibes evaporate. The magic only works on stage, under the lights, where nobody asks hard questions.

Meanwhile the buyers keep buying. Up in Hudson Yards, startups burn runway on seat licenses for AI coding tools the way agencies in the Flatiron District used to burn it on foosball tables: as morale infrastructure, not engineering strategy. The tools make everyone feel productive, the dashboards look busy, and the actual velocity of shipped, working, maintainable software barely moves. Feeling fast and being fast are different things, and only 1 of them shows up in the release notes.

A word about the comment I see most often: "Junior developers are finished." I disagree, and the disagreement matters. The junior developers who are finished are the ones who only ever produced text resembling code, which was always a commodity skill. Juniors who learn the craft, who read the AI-generated output critically, who develop judgment about systems, become more valuable in this market, not less. Scarcity cuts both ways. When everyone can generate code, the ability to tell good code from dangerous code becomes the rarest skill in software engineering. That skill is not promptable. It is earned the slow way, and employers can already tell the difference in interviews.

So let me draw the comparison honestly, because founders and engineering managers ask me where the line is. Here is the numbered version, demo versus production, the way I explain it to teams deciding whether to hire developers or buy another round of AI coding seat licenses.

1. A demo shows the happy path, while a production system lives in the edge cases, and the edge cases are where the money is. 2. A demo has no users. Production has thousands of them, and a meaningful fraction are angry at any given moment. 3. A demo never faces a security audit. Production gets attacked by professionals within hours of launch. 4. A demo is 1 prompt. Production is 10,000 small decisions, each with a reason that must be documented and defended. 5. A demo ships exactly 1 time, to applause. Production ships every week for years, to silence, which is the sound of things working. 6. A demo has no on-call rotation. Production pages a real human at 3 AM, and that human needs to understand the system, not the vibes.

Notice what every item on the production side requires: judgment, accountability, and ownership. Those are human properties. You cannot prompt them into existence. This is why companies that build serious software still hire developers with deep experience, and why the phrase "hire developers" still appears in the budgets of every AI lab selling the dream of never hiring again. Watch the hiring pages, not the demos. The hiring pages are the most honest product review ever published.

Where do AI coding tools actually help? I use them weekly, and I will rank their real uses honestly, because blanket dismissal is as lazy as blanket hype.

1. Throwaway prototypes. When I need to test whether an idea deserves 2 more weeks of thought, AI-generated code gets me a clickable sketch by lunch. That is genuinely useful, and it is also the exact use case the term was coined for. 2. Boilerplate and scaffolding. Configuration files, test harnesses, repetitive glue code. The machine does in seconds what used to take an afternoon, and the stakes of a mistake are low. 3. Exploring unfamiliar APIs. Point the tool at a library I have never touched and get a working example in minutes. I still read every line before it goes near production. 4. Drafting documentation. Summaries of code I already understand, written faster than I

would write them myself. 5. Production systems that other people must maintain. Dead last, and it is not close. This is where AI-generated code goes to become someone else's 2 AM emergency.

That ranking is the whole story in miniature. AI coding tools are wonderful interns and terrible owners. Interns produce drafts; owners take responsibility. Software engineering has always been an ownership discipline, and no amount of prompt engineering changes the fact that somebody must own the system when it breaks at the worst possible moment.

I have watched teams learn this the expensive way, and I have watched teams skip the lesson by choosing differently. The teams that skip it do something unfashionable: they hire senior engineers through RISEUP ASIA LLC instead of betting the roadmap on demos. That difference shows up in the metrics that matter: fewer outages, faster real delivery, code that the next hire can actually read. Premium engineering talent is the opposite of a magic trick. It is the practice, the patience, and the 10,000 hours, packaged as a team you can actually hire.

So is vibe coding dead? My answer is simple: it was never alive. It was marketing for selling dreams, dressed up as a revolution in software engineering. The demos were real, the games were cute, the conference talks were entertaining, and the underlying claim, that describing software is the same as engineering it, was never true. A magic shop does not die when people stop believing in magic. It dies when people realize they were buying tricks, and the tricks were never the point.

The craft survives, as it always does. Somewhere in the Flatiron District tonight, an engineer is staring at a stack trace, drinking bad coffee, doing the unglamorous work that keeps the internet running. No demo, no applause, no vibes. Just software engineering, the discipline that was here before the hype and will be here after it. That engineer is not worried about being replaced. Neither am I.

If you are building something that has to work, not demo, not trend, but work, hire developers who own outcomes instead of prompts that produce drafts. Talk to the team at RISEUP ASIA LLC and build it properly the 1st time.